

on it between the time we buy it and the time we register it. Here's a method that allows us to pass this incremental amount and add that value to the odometer reading:

```
class Car:
    --snip--

    def update_odometer(self, mileage):
        --snip--

    def increment_odometer(self, miles):
        """Add the given amount to the odometer reading."""
        self.odometer_reading += miles

❶ my_used_car = Car('subaru', 'outback', 2019)
   print(my_used_car.get_descriptive_name())

❷ my_used_car.update_odometer(23_500)
   my_used_car.read_odometer()

my_used_car.increment_odometer(100)
my_used_car.read_odometer()
```

The new method `increment_odometer()` takes in a number of miles, and adds this value to `self.odometer_reading`. First, we create a used car, `my_used_car` ❶. We set its odometer to 23,500 by calling `update_odometer()` and passing it 23_500 ❷. Finally, we call `increment_odometer()` and pass it 100 to add the 100 miles that we drove between buying the car and registering it:

```
2019 Subaru Outback
This car has 23500 miles on it.
This car has 23600 miles on it.
```

You can modify this method to reject negative increments so no one uses this function to roll back an odometer as well.

NOTE

You can use methods like this to control how users of your program update values such as an odometer reading, but anyone with access to the program can set the odometer reading to any value by accessing the attribute directly. Effective security takes extreme attention to detail in addition to basic checks like those shown here.

TRY IT YOURSELF

9-4. Number Served: Start with your program from Exercise 9-1 (page 162). Add an attribute called `number_served` with a default value of 0. Create an instance called `restaurant` from this class. Print the number of customers the restaurant has served, and then change this value and print it again.